

ABSTRACT

The traditional manual process of preparing lesson plans for an academic semester is time-consuming, prone to errors, and requires significant effort to ensure alignment with institutional policies, constraints, and curriculum requirements. This project presents an **AI-Powered Lesson Plan Automation System**, which aims to automate the generation of semester lesson plans by integrating advanced document processing, optical character recognition (OCR), artificial intelligence (AI) assisted academic recommendations, and constraint-based scheduling.

The system utilizes an automated workflow that accepts academic inputs in various formats—including PDF, DOCX, Excel, JPG, and PNG—such as the Academic Calendar, Faculty Timetable, Subject Syllabus, CO–PO Mapping, and Reference Materials. Using OpenCV and PaddleOCR for robust text extraction and dedicated parsers for digital documents, the system effectively structures this academic data. To ensure that scheduling remains accurate and respects real-world constraints, the system separates AI reasoning from schedule generation. The Groq-hosted LLM provides intelligent academic recommendations such as teaching methods, Bloom's taxonomy levels, assessment types, and practical activities, without handling deterministic scheduling tasks. The actual scheduling is governed by a Constraint Satisfaction Problem (CSP) algorithmic approach implemented in Python, which dynamically maps topics to available teaching periods while factoring in holidays, examination dates, and faculty workload.

Furthermore, the system accommodates template adaptation, detecting and mapping generated data to existing institutional lesson plan formats. A progress monitoring module allows faculty and HODs to track syllabus coverage, identify deviations, and update the schedule dynamically. Thus, this system significantly reduces administrative workload and enhances the quality of academic planning.

KEY TERMS: Artificial Intelligence, OCR, Constraint Satisfaction Problem, Deep Learning, Document Processing, Educational Technology

TABLE OF CONTENT

CHAPTER TITLE NO.	PAGE NO.
ABSTRACT	iv
LIST OF TABLES	vii
LIST OF FIGURES	viii
LIST OF ABBREVIATIONS	ix
1 INTRODUCTION	1
1.1 OVERVIEW	1
1.2 OBJECTIVE	3
1.3 SCOPE	3
2 LITERATURE SURVEY	4
3 IDEATION & PROPOSED SOLUTION	6
3.1 EXISTING SYSTEM	6
3.1.1 DRAWBACKS	6
3.2 PROPOSED SYSTEM	7

3.2.1 ADVANTAGES	8
4 SYSTEM REQUIREMENT	9
4.1 HARDWARE REQUIREMENT	9
4.2 SOFTWARE REQUIREMENT	9
5 SYSTEM DESIGN	13
5.1 ARCHITECTURE DIAGRAM	13
5.2 DATA FLOW DIAGRAM	14
6 MODULES	16
6.1 DOCUMENT PROCESSING	16
6.2 LESSON PLAN CONFIGURATION & TEMPLATE ADAPTATION	17
6.3 CSP SCHEDULING & AI RECOMMENDATIONS	18
6.4 PROGRESS MONITORING	18
7 SYSTEM ORGANIZATION	19
7.1 USE CASE DIAGRAM	19
7.2 CLASS DIAGRAM	20

7.3	ACTIVITY DIAGRAM	21
7.4	SEQUENCE DIAGRAM	22
8	DATABASE AND REST API STRUCTURE	23
8.1	DATABASE STRUCTURE	23
8.2	REST API STRUCTURE	24
9	IMPLEMENTATION AND SECURITY	25
9.1	FRONTEND IMPLEMENTATION	25
9.2	BACKEND IMPLEMENTATION	29
9.3	SECURITY FEATURES	31
9.4	RESULT	33
10	TESTING	35
10.1	TEST CASES	35
10.2	USER ACCEPTANCE TESTING	35
10.2.1	DEFECT ANALYSIS	36
10.2.2	TEST CASE ANALYSIS	36

11	CONCLUSION AND FUTURE WORK	37
	11.1 CONCLUSION	37
	11.2 FUTURE ENHANCEMENTS	37

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
1	HARDWARE REQUIREMENTS	9
2	DEFECT ANALYSIS	36
3	TEST CASE ANALYSIS	36

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
1	INPUT DOCUMENTS	7
2	ARCHITECTURE OF AI AND SCHEDULING	7
3	SYSTEM ARCHITECTURE DIAGRAM	13
4	DATA FLOW DIAGRAM	14
5	USE CASE DIAGRAM	19
6	CLASS DIAGRAM	20
7	ACTIVITY DIAGRAM	21
8	SEQUENCE DIAGRAM	22

LIST OF ABBREVIATIONS

AI- ARTIFICIAL
INTELLIGENCE

OPTICAL
OGRCHARACTER
RECOGNITION

CONSTRAINT
CSP SATISFACTION
PROBLEM

LARGE
LLMLANGUAGE
MODEL

CQ COURSE
OUTCOME

PQ PROGRAM
OUTCOME

JSON
JWTWEB
TOKEN

UI- USER
INTERFACE

APPLICATION
API PROGRAMMING
INTERFACE

HEAD
HODOF
DEPARTMENT

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

In recent years, the adoption of digital technologies in educational institutions has increased tremendously. Despite this, administrative and academic planning processes, such as the preparation of a semester lesson plan, remain heavily dependent on manual effort. A lesson plan dictates the structure of teaching for the entire semester, incorporating topics to be covered, teaching methodologies, assessment strategies, and alignment with course outcomes. The manual generation of such a document is repetitive and prone to scheduling conflicts and errors.

The AI-Powered Lesson Plan Automation System introduces a modern technological approach to overcome these challenges. The system is designed to automate semester lesson plan preparation by integrating document processing, Optical Character Recognition (OCR), AI-assisted academic recommendations, constraint-based scheduling, template adaptation, and syllabus progress monitoring. By processing source documents such as the Academic Calendar, Faculty Timetable, Subject Syllabus, CO–PO Mapping, and Reference Materials, the system structures necessary information efficiently.

Supported formats include PDF, DOCX, Excel, JPG, and PNG. Document processing is carried out using robust libraries like PyMuPDF, python-docx, and OpenCV combined with PaddleOCR for image-based documents. Once structured data is obtained, the system creates the schedule using a deterministic approach.

A core technical concept of this system is the strict separation between AI reasoning and schedule generation. The Large Language Model (LLM) powered by the Groq API is dedicated entirely to generating high-quality academic recommendations. These include teaching methods, Bloom's Taxonomy levels, learning outcomes, assessments, practical activities, quizzes, assignments, revision sessions, and short teaching notes. The LLM is strictly prohibited from making deterministic scheduling decisions such as allocating dates, periods, or considering holidays. Instead, a Python-based Constraint Satisfaction Problem (CSP) scheduling algorithm guarantees an error-free timetable.

The application leverages a modern technology stack to deliver a seamless user experience. The frontend is built using React.js and Bootstrap 5, handling routing via React Router and API requests via Axios. The backend is a fast, asynchronous Python server

powered by FastAPI, interacting with a MongoDB database. Security is robust, utilizing JWT Authentication. Report generation features provide the final output in customizable Excel, Word, or PDF formats using openpyxl, python-docx, and ReportLab.

1.2 OBJECTIVE

The main Objective of the project is: * To completely automate the semester lesson plan preparation by accurately processing and extracting academic data from various unstructured and semi-structured input documents. * To separate the scheduling logic from AI generation by utilizing a Constraint Satisfaction Problem (CSP) algorithm for precise date and period allocation. * To utilize a Groq-hosted LLM to generate intelligent academic recommendations such as Bloom's Taxonomy level, teaching method, and assessment strategies for each topic. * To support existing institutional lesson plan templates, automatically detecting their structure and seamlessly mapping generated data into the preferred format.

1.3 SCOPE

The proposed model in this project work can be extended to an integrated academic management suite. The entire system is designed with a scalable microservices-like backend architecture using FastAPI, making it easily adaptable. The dynamic template adaptation and robust constraint scheduling logic can be integrated into Enterprise Resource Planning (ERP) systems or Learning Management Systems (LMS). Furthermore, the document processing modules utilizing OpenCV and PaddleOCR can be leveraged for analyzing other institutional documents.

The versatile nature of the constraint-based scheduling model allows for seamless modifications to accommodate varying institutional policies, dynamic holiday schedules, and unexpected timetable changes. The scalability of the AI module through the Groq API opens up possibilities for integration with more advanced predictive academic analytics in the future.

CHAPTER 2

LITERATURE SURVEY

Educational technology has witnessed immense growth, specifically in the domains of scheduling and artificial intelligence applications in academic planning.

Traditional timetabling and scheduling are classic examples of the Constraint Satisfaction Problem (CSP). Numerous researchers have proposed algorithms such as genetic algorithms, simulated annealing, and backtracking search to solve scheduling conflicts. However, these systems often require highly structured, manually entered data. In a typical academic environment, constraints such as faculty availability, working days, and subject hours are distributed across multiple documents, requiring a unified system to parse and interpret them before scheduling can commence.

The integration of Optical Character Recognition (OCR) in educational administration has proven effective for digitizing records. Tools like Tesseract and PaddleOCR have been evaluated for extracting information from complex tables such as timetables and academic calendars. Studies show that preprocessing techniques—such as deskewing, grayscale conversion, and contrast enhancement using OpenCV—significantly improve the accuracy of data extraction from physical documents or scanned images.

Recent advancements in Large Language Models (LLMs) have opened new avenues for content generation in education. LLMs are highly proficient at mapping concepts to Bloom's Taxonomy, suggesting engaging teaching methods, and generating relevant assessment questions. However, literature highlights a critical limitation of LLMs: their propensity for "hallucination" and inability to reliably solve complex, multi-constraint deterministic problems like calendar scheduling. Researchers advocate for a hybrid approach where an LLM is utilized exclusively for semantic reasoning and content generation, while traditional algorithmic solvers handle the deterministic constraints.

This project addresses these gaps by implementing a hybrid architecture. It employs OpenCV and PaddleOCR for robust data extraction, a Python-based CSP solver for precise timetable allocation, and a Groq-hosted LLM for intelligent academic recommendations, delivering a comprehensive, automated lesson planning solution.

CHAPTER 3

IDEATION & PROPOSED SOLUTION

3.1 EXISTING SYSTEM

In most educational institutions, the preparation of lesson plans is a manual, tedious, and time-consuming process. At the beginning of each semester, faculty members are required to gather multiple documents: the Academic Calendar published by the university, the Faculty Timetable provided by the department, the Subject Syllabus, and CO-PO mapping guidelines.

The faculty must manually calculate working days, exclude holidays and examination dates, and map the syllabus topics to available teaching periods. They must also independently decide on teaching methodologies, Bloom's Taxonomy levels, and appropriate assessments. Furthermore, this data must be painstakingly formatted into an institution-specific template (often an Excel or Word file). Any unexpected changes to the timetable or academic calendar require a complete manual recalculation of the subsequent schedule.

3.1.1 DRAWBACKS

- **High Manual Effort:** Faculty members spend significant administrative time cross-referencing calendars, timetables, and syllabi.
- **Error-Prone Scheduling:** Manual calculation of dates and periods frequently leads to scheduling conflicts or failure to meet the required teaching hours.
- **Lack of Standardization:** Teaching methods and assessment strategies can vary widely in quality and alignment with the syllabus without intelligent guidance.
- **Inflexibility:** When holidays are declared unexpectedly or sessions are canceled, updating the lesson plan manually is extremely cumbersome.
- **Template Rigidity:** Formatting data into specific institutional templates manually is tedious.

3.2 PROPOSED SYSTEM

The proposed **AI-Powered Lesson Plan Automation System** eliminates these drawbacks by automating the entire lifecycle of lesson plan generation. The system operates by processing the uploaded documents (Academic Calendar, Timetable, Syllabus, etc.) using OCR and parsing techniques to extract structured data.

A deterministic CSP algorithm then takes over to calculate the exact dates, periods, and teaching hours for each topic, strictly adhering to the faculty's availability and the academic calendar's constraints. Concurrently, the Groq-hosted LLM processes the syllabus topics to generate tailored academic recommendations. Finally, the system maps this comprehensive dataset into the institution's existing lesson plan template, generating a polished document ready for review and export.

3.2.1 ADVANTAGES

- **Automated Academic Data Processing:** Eliminates the need for manual data entry by extracting structured data from multi-format input documents.
- **Constraint-Aware Scheduling:** Guarantees accurate date and period allocation by intelligently skipping holidays, exams, and identifying correct timetable slots.
- **Separation of Concerns:** By using a CSP algorithm for scheduling and an LLM exclusively for semantic reasoning, the system ensures both deterministic accuracy and high-quality AI suggestions.
- **Institution-Specific Template Adaptation:** Seamlessly adapts the generated lesson plan to match the existing format, columns, and layout of the institution.
- **Dynamic Rescheduling & Progress Monitoring:** Tracks completed sessions and easily regenerates the remaining schedule if deviations occur.

3.2.2 LIMITATIONS

- OCR accuracy depends heavily on the quality, contrast, and resolution of the input document.
- AI-generated recommendations require faculty validation, as automated reasoning may occasionally misalign with nuanced pedagogical intent.
- Major changes in institutional policies or document formats may require configuration updates or parsing logic modifications.
- Scheduling quality is directly dependent on the completeness and correctness of timetable and academic-calendar data. Human validation is still essential.

CHAPTER 4

SYSTEM REQUIREMENT

4.1 HARDWARE REQUIREMENTS

Component	Minimum Requirement
Processor	64-bit, multi-core, 2.5 GHz minimum
RAM	8 GB for evaluation, 16 GB for production
Hard Disk	20 GB free space

TABLE 1: HARDWARE REQUIREMENTS

4.2 SOFTWARE REQUIREMENTS

- **Frontend:** React.js, Bootstrap 5, Axios, React Router
- **Backend:** Python 3.9+, FastAPI
- **Database:** MongoDB
- **AI & API:** Groq API (Groq-hosted LLM)
- **Image Processing & OCR:** OpenCV, PaddleOCR
- **Document Parsing & Reporting:** PyMuPDF, pdfplumber, python-docx, openpyxl, ReportLab
- **Environment:** VS Code, Node.js

4.3 FUNCTIONAL & NON-FUNCTIONAL REQUIREMENTS

Functional Requirements: * The system shall allow faculty to upload Academic Calendars, Timetables, Syllabi, CO-PO mappings, and reference materials. * The system shall parse and extract structured data from PDF, DOCX, Excel, JPG, and PNG files. * The system shall schedule topics using a CSP algorithm without conflicts. * The system shall generate academic recommendations using the Groq LLM. * The system shall map data to a provided institutional template and allow export in PDF, Excel, and Word formats.

Non-Functional Requirements: * **Security:** The system shall protect user data using JWT Authentication and role-based access control (Admin, HOD, Faculty). * **Performance:** OCR and AI recommendations shall process within acceptable timeframes to maintain a responsive user experience. * **Reliability:** The deterministic scheduling must produce highly reliable calendar mappings.

CHAPTER 5

SYSTEM DESIGN

5.1 ARCHITECTURE DIAGRAM

The system architecture is structured into multiple layers to ensure modularity and separation of concerns. The frontend communicates with the FastAPI backend via secure REST endpoints. The backend orchestrates document processing, scheduling, AI generation, and database interactions.

```
text Faculty ↓ React.js Frontend ↓ FastAPI Backend ↓
Authentication Layer (JWT) ↓ Document Processing Layer ■■■■
OpenCV (Image enhancement) ■■■■ PaddleOCR (Text extraction)
■■■■ PyMuPDF / pdfplumber (PDF) ■■■■ python-docx (Word) ■■■■
openpyxl (Excel) ↓ Academic Data Layer (Structuring extracted
info) ↓ MongoDB (Data storage) ↓ Scheduling Layer ■■■■ CSP
Algorithm (Deterministic mapping of dates & periods) ↓ AI
Layer ■■■■ Groq LLM (Academic recommendations) ↓ Template /
Progress / Report Layer ↓ Final Lesson Plan (PDF/Excel/Word)
```

Figure 3: System Architecture Diagram

5.2 DATA FLOW DIAGRAM

The data flow highlights how information transitions from raw unstructured documents to a final formatted lesson plan.

```
text [INPUT DOCUMENTS] ↓ [DOCUMENT UPLOAD & VALIDATION] ↓
[IMAGE PROCESSING & OCR / TEXT PARSING] ↓ [STRUCTURED DATA
EXTRACTION] ↓ [PREVIEW & CORRECTION] ↓ (Data valid?) --- NO
---> [FACULTY CORRECTION] | YES ↓ [CSP SCHEDULING ALGORITHM] ↓
[GROQ LLM RECOMMENDATIONS] ↓ [TEMPLATE ADAPTATION] ↓ [FINAL
LESSON PLAN GENERATION]
```

Figure 4: Data Flow Diagram

CHAPTER 6

MODULES

6.1 DOCUMENT PROCESSING

This module handles the extraction of text from various file formats. For digital documents, dedicated parsers like PyMuPDF and pdfplumber are used for PDFs, python-docx for Word, and openpyxl for Excel.

For image-based documents, OpenCV is utilized for pre-processing. The pipeline performs grayscale conversion, deskewing, noise removal, contrast enhancement, sharpening, and cropping to prepare the image. The enhanced image is then processed by PaddleOCR, which extracts the text data accurately. The extracted data includes Semester details, Holidays, Faculty timetable slots, Course Codes, Syllabus topics, and CO-PO mappings.

6.2 LESSON PLAN CONFIGURATION & TEMPLATE ADAPTATION

Faculty members can configure the required lesson plan columns, such as Sl. No., Day, Date, Period, Lesson Topic, Unit, Teaching Hours, CO, PO, Bloom Level, Teaching Method, Assessment, etc. The lesson plan layout can be configured as Unit-wise, Week-wise, or Continuous.

The Template Adaptation feature allows the system to analyze an uploaded institutional template. It identifies required columns, column order, unit structure, and institution-specific fields. The generated data is then dynamically mapped to this detected structure, ensuring the final export strictly follows the institution's format.

6.3 CSP SCHEDULING & AI RECOMMENDATIONS

A core design principle is the separation of scheduling and AI. The **CSP Scheduling Module** acts on the structured academic data. It identifies available teaching periods, allocates topics based on required hours, skips holidays and exams, and calculates exact dates and periods. It handles conflict resolution and can dynamically push canceled sessions to the next valid period.

Concurrently, the **Groq AI Module** processes the syllabus topics. The LLM generates recommendations for Teaching Method, Bloom's Taxonomy Level, Learning Outcome, Assessment, Practical Activity, Quizzes, and References. The LLM is restricted

from generating any dates, periods, or timetables.

6.4 PROGRESS MONITORING

This module tracks the execution of the lesson plan throughout the semester. It monitors planned versus completed sessions, calculates syllabus coverage percentages, and identifies deviations. If a class is missed, the module allows faculty to update the remaining schedule dynamically without affecting past entries.

CHAPTER 7

SYSTEM ORGANIZATION

7.1 USE CASE DIAGRAM

The system features distinct roles for Faculty, HOD, and Admin. The Use Case diagram describes the core interactions of these actors with the system.

- **Faculty:** Upload documents, Correct extracted data, Configure lesson plan, Generate plan, Edit, Review, Approve, Export, Monitor progress.
- **HOD:** Approve lesson plans, View faculty workload, Monitor syllabus progress across the department.
- **Admin:** Manage faculty, departments, and subjects, View all lesson plans, Approve/Reject.

7.2 CLASS DIAGRAM

The class diagram conceptualizes the backend structure. Key conceptual classes include: * **User:** Attributes for credentials and roles. * **AcademicDocument:** Base interface for parsing logic across different formats. * **CSPScheduler:** Encapsulates methods for resolving constraints, mapping topics, handling holidays. * **GroqAIClient:** Handles prompt construction, API requests, and response parsing for recommendations. * **TemplateAdapter:** Logic for mapping generic lesson plan output to specific institutional formats.

7.3 ACTIVITY DIAGRAM

The activity diagram depicts the sequential flow of generating a lesson plan: 1. Faculty logs in and uploads source documents. 2. System performs OCR / parsing operations based on the file type. 3. System structures the extracted text into academic data. 4. Faculty previews and corrects data. 5. System applies the CSP scheduling algorithm. 6. System fetches semantic AI recommendations from Groq. 7. System applies template mapping. 8. Faculty reviews and exports the final document.

7.4 SEQUENCE DIAGRAM

The sequence diagram illustrates the interaction between the User (Faculty), the FastAPI Backend, the MongoDB Database, and external services (Groq API, PaddleOCR). The backend handles requests asynchronously, dispatching intensive OCR and AI network

tasks efficiently, and finally returning the structured, verified result to the user interface.

CHAPTER 8

DATABASE AND REST API STRUCTURE

8.1 DATABASE STRUCTURE

The system utilizes MongoDB, a NoSQL database, to store flexible document structures. Important collections include: * **users**: Stores credentials, roles (Admin, HOD, Faculty), and profile details. * **faculty**: Contains detailed faculty profiles, department links, and workload. * **subjects**: Stores subject codes, names, and department associations. * **academic_calendar**: Stores semester start/end dates, working days, holidays, and exam schedules. * **faculty_timetable**: Stores allocated subjects, days, periods, and sections. * **syllabus**: Contains units, topics, teaching hours, and course outcomes. * **co_po_mapping**: Stores the matrix correlating Course Outcomes with Program Outcomes. * **lesson_plan_config**: Stores faculty preferences for columns and layout. * **lesson_plans**: The core collection storing the generated schedules, AI recommendations, and completion status. * **reports**: Stores metadata for exported PDF/Excel/Word files. * **uploads**: Tracks file metadata and processing status. * **notifications**: Stores system alerts and approval requests.

8.2 REST API STRUCTURE

The FastAPI backend exposes modular REST endpoints for interaction:

- **/auth**: POST /login, POST /register
- Purpose: User authentication and issuing JSON Web Tokens.
- **/upload**: POST /upload/calendar, POST /upload/timetable, POST /upload/syllabus
- Purpose: Accepts files, triggers processing, requires JWT auth.
- **/parser & /ocr**: GET /parser/status/{task_id}
- Purpose: Polls the status of asynchronous background parsing jobs.
- **/lessonplan**: POST /lessonplan/generate, GET /lessonplan/{id}
- Purpose: Aggregates structured data, triggers the CSP algorithm, fetches Groq AI insights, and saves the lesson plan.
- **/report**: GET /report/export/{id}?format=pdf
- Purpose: Initiates the Template Adaptation module and streams the final document (Excel/Word/PDF) back to the user.
- **/admin**: Routes for managing faculty and departments.

CHAPTER 9

IMPLEMENTATION AND SECURITY

9.1 FRONTEND IMPLEMENTATION

The frontend is implemented using React.js and styled with Bootstrap 5. Key pages include: * **Login & Dashboard:** Provides role-based access and an overview of current tasks. * **Upload Documents & OCR Preview:** Interfaces for uploading files and a validation screen where faculty can correct any OCR errors before saving. * **Lesson Plan Configuration:** A dynamic form to select required columns (Sl. No., Day, Date, Period, Topic, CO, PO, Bloom Level, Teaching Method) and desired layout. * **Generate & Preview Lesson Plan:** Displays the final generated lesson plan in a comprehensive table, allowing manual edits to AI suggestions. * **Reports & Profile:** Pages for downloading finalized plans and managing user details.

9.2 BACKEND IMPLEMENTATION

The FastAPI backend orchestrates several complex services concurrently: * **Document Parsing:** PyMuPDF and python-docx extract text, which is then structured using regex and heuristic mapping into JSON format. Openpyxl is utilized for direct cell data mapping. * **CSP Scheduling:** Implemented entirely in Python, the scheduling algorithm iterates over syllabus topics, deducting required hours. It queries the timetable for the next available period, checks against the holiday list, and assigns a specific date and period. If a constraint is violated, it logically steps forward to the next valid slot, regenerating segments dynamically. * **Groq Integration:** The backend constructs a structured prompt containing the syllabus topic and Course Outcome, querying the Groq API. The response is parsed to extract Teaching Method, Bloom Level, Assessment, and Activity, appending them to the scheduled entry while maintaining strict isolation from date calculations.

9.3 SECURITY FEATURES

- **Authentication:** Implemented securely using JSON Web Tokens (JWT). All protected routes require a valid Bearer token.
- **Authorization:** Role-based access control prevents Faculty from accessing Admin/HOD routes, ensuring strict data boundaries.
- **Input Validation:** Pydantic models in FastAPI strictly validate all incoming request payloads, mitigating injection risks.

- **Secrets Management:** Environment variables are strictly utilized to secure database URIs, Groq API keys, and JWT secret keys. Error handling prevents tracebacks from leaking sensitive environment configurations to the frontend.

CHAPTER 10

TESTING

10.1 TEST CASES

Test Case ID	Module	Input	Expected Result	Actual Result	Status
TC_01	Upload	PDF Syllabus File	File uploaded successfully, triggers parsing	File uploaded and parsed	Pass
TC_02	OCR	Image of Timetable	Extracts subjects and periods accurately	Text extracted accurately	Pass
TC_03	Scheduling	Calendar + Timetable + 5 Topics	Assigns exact dates skipping holidays	Dates assigned without conflicts	Pass
TC_04	AI Integration	Topic: "Machine Learning"	Returns Teaching Method & Bloom Level	Returned relevant recommendations	Pass

TC_05	Export	Generated Lesson Plan Data	Produces formatted Excel matching template	Excel file generated correctly	Pass
-------	--------	----------------------------	--	--------------------------------	------

10.2 USER ACCEPTANCE TESTING

User Acceptance Testing (UAT) involves deploying the system to a subset of faculty members. Scenarios include uploading real departmental timetables and institutional calendars to verify that the scheduling engine accurately maps dates over a 14-week semester without missing required teaching hours. Faculty evaluate the AI-generated recommendations for relevance and pedagogical accuracy.

10.2.1 DEFECT ANALYSIS

During initial testing phases, defects related to OCR accuracy on low-contrast images and minor edge cases in scheduling around consecutive holidays were identified and resolved by enhancing OpenCV preprocessing and refining the CSP algorithm constraints.

Resolution	Severity 1	Severity 2	Severity 3	Severity 4	Severity 5
By Design	0	0	1	0	1
Duplicate	1	3	2	2	8
External	2	3	0	0	5
Fixed	4	4	4	4	16
Total	7	10	7	6	30

TABLE 2: DEFECT ANALYSIS

10.2.2 TEST CASE ANALYSIS

Section	Total Cases	Not Tested	Fail	Pass
Frontend UI	5	0	0	5
API & Auth	5	0	0	5
Document Parsing	4	0	0	4
Scheduling	4	0	0	4
AI Generation	3	0	0	3
Export & Reporting	4	0	0	4

TABLE 3: TEST CASE ANALYSIS

CHAPTER 11

CONCLUSION AND FUTURE WORK

11.1 CONCLUSION

The AI-Powered Lesson Plan Automation System successfully addresses the inefficiencies associated with manual academic planning. By integrating OCR and document parsing, the system efficiently extracts structured data from disparate academic documents. The critical architectural decision to separate AI reasoning from scheduling ensures that the generated lesson plans are both deterministically accurate (respecting holidays and timetables) and pedagogically rich (featuring Groq-powered recommendations). Furthermore, the template adaptation and progress monitoring modules deliver a tailored, dynamic solution that significantly reduces administrative workload and enhances institutional productivity.

11.2 FUTURE ENHANCEMENTS

Future extensions of the system include: * **LMS and ERP Integration:** Direct synchronization of the generated lesson plan and progress data with existing institutional management systems. * **Advanced Academic Analytics:** Predictive analysis of syllabus completion rates based on historical progression data. * **Automated Notification System:** Email and SMS alerts for HODs and faculty regarding lagging syllabus coverage or schedule deviations. * **Mobile Application:** A dedicated mobile interface for faculty to quickly update progress and view their daily schedule on the go. * **Multi-language Document Processing:** Enhancing the OCR and AI models to support lesson plan generation in regional languages.